# Cursor on Target (CoT) — cot-sample

# Cursor on Target (CoT) — overview and sample generators

Copyright (c) 2026 Rally Tactical Systems, Inc.
All rights reserved.

This directory contains small **sample applications** that emit **CoT event XML** (UTF-8) for testing, documentation, and integration experiments. They do not implement a full tactical client or server; they only generate a **minimal, valid-shaped** event document you can send over UDP/TCP or hand to other tools.

---

## What is Cursor on Target?

**Cursor on Target (CoT)** is an open, XML-centric messaging pattern used heavily in the **Team Awareness Kit (TAK)** ecosystem and related C2 / situational-awareness systems. A CoT **event** describes *what* something is (type), *where* it is (point), *when* it applies (time / stale), and *how* the data was produced (`how`), with optional **detail** payloads (contact, remarks, custom tags, etc.).

Typical uses:

- **Position reporting** — ownship / friendly / track updates.
- **Markers and annotations** — points, routes, shapes (often via type strings and detail extensions).
- **Gateway formats** — systems wrap domain payloads inside `<detail>` (e.g. JSON or XML fragments) for consumers that understand those extensions.

CoT is **not** one monolithic RFC: operational behavior is defined by **community practice**, **TAK product expectations**, and **type catalogs** (e.g. MIL-STD-2525-style type strings). Always validate against the **specific consumer** (ATAK, WinTAK, TAK Server, custom bridge, etc.) you target.

---

# External references

| Resource | URL |
|---|---|
| TAK program (official portal) | https://tak.gov/ |
| TAK Product Center (open-source hubs, Server, clients, tooling) | https://github.com/TAK-Product-Center |
| Civilian ATAK (ATAK-CIV) — useful for wire-format and behavior discussions | https://github.com/deptofdefense/AndroidTacticalAssaultKit-CIV |
| Wikipedia — high-level summary of CoT | https://en.wikipedia.org/wiki/Cursor_on_Target |
| Protocol Buffers (language-neutral serialization used by many TAK protobuf paths) | https://protobuf.dev/ |
| `takproto` (Python helpers / protobuf ↔ XML discussion in ecosystem) | https://pypi.org/project/takproto/ |

Historical / archival note: older **MITRE CoT developer** materials have circulated as PDFs (e.g. via NPS and other mirrors). Treat them as **background**; for product behavior, prefer **current TAK** documentation and the **open-source TAK** repositories above.

---

# CoT XML shape (what these samples emit)

Both sample programs generate a **single root `<event>`** with **`version="2.0"`**, one **`<point />`**, and a **`<detail>`** block. Timestamps are **UTC** with **millisecond** precision (`YYYY-MM-DDTHH:MM:SS.sssZ`).

### `<event>` attributes

| Attribute | Required for interchange | In these samples | Notes |
|---|---|---|---|
| `version` | Yes (consumers expect a version) | Always `2.0` | |
| `uid` | Yes | `--uid` (default `cot-sample-uid`) | Stable unique id for the event instance. |
| `type` | Yes | `--type` (default `a-f-G-U-C`) | Often a **MIL-STD-2525** style |

| Attribute | Required for interchange | In these samples | Notes |
|---|---|---|---|
| | | | string; meaning is catalog-specific. |
| `time` | Yes | Current UTC time at generation | |
| `start` | Yes | Same as `time` in samples | Some pipelines treat start vs time differently. |
| `stale` | Yes | `time` + `--stale-offset-secs` (default **300** s) | Receivers may drop events after stale. |
| `how` | Yes | `--how` (default `h-e`) | Provenance / processing hint (e.g. human-entered, machine). |

## `<point>` attributes

| Attribute | Required for interchange | In these samples | Notes |
|---|---|---|---|
| `lat`, `lon` | Yes | `--lat`, `--lon` (defaults `0.0`) | Decimal degrees. |
| `ce` | Often present | `--ce` (default `9999999`) | Circular error / horizontal uncertainty; **`9999999`** commonly means *unknown*. |
| `hae` | Often present | `--hae` (default `0`) | Height above ellipsoid (meters). |
| `le` | Often present | `--le` (default `9999999`) | Linear error; **`9999999`** often *unknown*. |

### `<detail>` contents (optional in schema, used here for demos)

| Element | In these samples | Notes |
|---|---|---|
| `<contact callsign="…" [endpoint="…"] />` | If `--callsign` set; `--endpoint` optional | Common for SA clients. |
| `<remarks>… </remarks>` | Default unless `--no-remarks` | Free text; samples place **compact JSON** here by default. |
| Custom tags from `--detail-json` | Optional | Each **top-level string** property becomes `<key>value</key>`. |

---

## XML declaration "formats" (`--format`)

The samples support two **declaration** styles (both still **CoT 2.0 XML**):

| Value | XML declaration |
|---|---|
| `original` | `<?xml version="1.0" encoding="UTF-8"?>` |
| `new` (default) | `<?xml version="1.0" encoding="UTF-8" standalone="yes"?>` |

Some mobile / ATAK-style emitters use `standalone="yes"`; other gateways omit it. Pick what matches your consumer tests.

---

## Sample application: `generate_cot_payload.py`

**Language:** Python 3 (stdlib only: `argparse`, `json`, `xml.etree.ElementTree`, etc.)

**Purpose:** Build one CoT XML document and write **UTF-8 bytes** to **stdout** or `--output-file`.

### Build / run

```
python3 generate_cot_payload.py --help
python3 generate_cot_payload.py --format new --pretty
```

### Command-line options

| Option | Description |
|---|---|
| `--format original\|new` | XML declaration style (default: `new`). |

| Option | Description |
|---|---|
| `--payload-json JSON` | JSON object → serialized compact JSON inside `<remarks>`. Mutually exclusive with `--payload-file`. |
| `--payload-file PATH` | UTF-8 file contents → `<remarks>`. |
| `--no-remarks` | Omit `<remarks>` entirely. |
| `--uid`, `--type`, `--how` | Event attributes (defaults: `cot-sample-uid`, `a-f-G-U-C`, `h-e`). |
| `--lat`, `--lon`, `--ce`, `--hae`, `--le` | Point attributes (see defaults above). |
| `--stale-offset-secs N` | Seconds after "now" for `stale` (default `300`). |
| `--callsign`, `--endpoint` | Optional `<contact … />` inside `<detail>`. |
| `--detail-json JSON` | JSON object: each **string** top-level value becomes a child element under `<detail>`. |
| `--pretty` | Pretty-print: nested elements, **one XML attribute per line**. |
| `--indent TEXT` | Indentation per level for `--pretty` (default two spaces). |
| `-o`, `--output-file PATH` | Write bytes to file instead of stdout. |

**`--payload-json` validation:** the argument must be valid JSON; it is **re-serialized** to a compact single-line string before XML escaping.

**`--detail-json` validation:** must be a JSON **object**; only **string** values produce extra elements.

## Sample: pretty-printed output (`--pretty`)

Timestamps change every run; structure and indentation match the tool.

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<event
  version="2.0"
  uid="cot-sample-uid"
  type="a-f-G-U-C"
  time="2026-05-16T12:00:00.000Z"
  start="2026-05-16T12:00:00.000Z"
  stale="2026-05-16T12:05:00.000Z"
  how="h-e">
  <point
    lat="0.0"
    lon="0.0"
    ce="9999999"
    hae="0"
    le="9999999" />
  <detail>
    <contact
      callsign="ALPHA-1"
```

```
      endpoint="192.168.1.10:8087:tcp" />
    <remarks>
      {"note":"CoT sample payload","source":"cot-sample"}
    </remarks>
    <missionName>Training</missionName>
  </detail>
</event>
```

Example command for the above shape (minus your own timestamps):

```
python3 generate_cot_payload.py --pretty \
  --callsign ALPHA-1 \
  --endpoint "192.168.1.10:8087:tcp" \
  --detail-json '{"missionName":"Training"}'
```

### Sample: compact one-line output (default)

Omit `--pretty` to get a single-line document suitable for piping over the network:

```
python3 generate_cot_payload.py --format original --no-remarks
```

---

## Sample application: `generate_cot_payload` (C++11)

**Language:** C++11, **no third-party libraries** (only the C++ standard library and system `gmtime_r` / `gmtime_s`).

**Purpose:** Same as the Python tool: emit one CoT XML document to **stdout** or `-o` / `--output-file`.

### Build

```
make
# produces ./generate_cot_payload
```

`make clean` removes the binary.

### Command-line options

The C++ tool accepts the **same flags** as Python, with these notes:

| Option | Notes |
|---|---|
| `--pretty` **or** `--human-readable` | Both enable pretty-print (Python only documents `--pretty`). |
| `--payload-json STR` | Passed through as **remarks text** without JSON parse/re-serialize (caller supplies exact UTF-8 string). |
| `--detail-json` | JSON object; **string** values → elements; non-string values skipped (nested |

| Option | Notes |
|---|---|
| | objects/arrays are skipped via a small internal skipper). |

Use `-h` / `--help` for a short usage summary.

### Sample: pretty-printed output

Same layout as Python (`--pretty`, one attribute per line). Example:

```
./generate_cot_payload --pretty --callsign BRAVO
```

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<event
  version="2.0"
  uid="cot-sample-uid"
  type="a-f-G-U-C"
  time="2026-05-16T12:00:00.000Z"
  start="2026-05-16T12:00:00.000Z"
  stale="2026-05-16T12:05:00.000Z"
  how="h-e">
  <point
    lat="0.0"
    lon="0.0"
    ce="9999999"
    hae="0"
    le="9999999" />
  <detail>
    <contact
      callsign="BRAVO" />
    <remarks>
      {"note":"CoT sample payload","source":"cot-sample"}
    </remarks>
  </detail>
</event>
```

---

# UDP multicast transmission

CoT XML is often carried as **plain UDP**: one **UTF-8** datagram per event (no extra framing unless your stack adds one). Keep the XML **compact** on the wire (`--pretty` is for humans only; it increases size and can push you toward MTU issues on Wi-Fi).

### ATAK default address and port

The most common **multicast group / UDP port** pairing used by **ATAK** for this style of traffic is:

| Setting | Value |
|---|---|
| Multicast group | `239.1.2.1` |
| UDP port | `6969` |

Treat this as a **widely used default**, not a universal constant. Different missions, TAK Server configs, plugins, or product variants may use other groups or ports. Confirm under the client's **network / multicast / TAK Reticulum** (or equivalent) settings for the build you are testing.

## Send one CoT event to multicast

Pipe **compact** XML from either generator to UDP multicast `239.1.2.1:6969`. **`socat`** is the most portable when you need extra socket options; **`nc`** is often enough for simple LAN tests.

**With `socat`:**

```
python3 generate_cot_payload.py | socat - UDP4-DATAGRAM:239.1.2.1:6969

./generate_cot_payload | socat - UDP4-DATAGRAM:239.1.2.1:6969
```

Install `socat` if needed (e.g. `brew install socat` on macOS, or your Linux distro package).

**With netcat (`nc`)** — multicast is still a UDP destination, so many `nc` builds can send one datagram per run. **macOS `/usr/bin/nc` is not OpenBSD `nc`:** do **not** use `-N` there; Apple's `nc` uses `-N` for unrelated TCP timeout options and fails with errors like `invalid tcp adaptive write timeout value`. Use **`-w`** instead so the process exits after the send.

```
python3 generate_cot_payload.py | nc -u -w 1 239.1.2.1 6969

./generate_cot_payload | nc -u -w 1 239.1.2.1 6969
```

- **`-u`** — UDP.
- **`-w 1`** — wait at most **1 second** after stdin closes, then quit (portable way to avoid a hung pipeline on macOS and many Linux images).

On some **OpenBSD** `nc` builds only, **`-N`** means "shutdown network after stdin EOF"; that flag is **safe only where documented** for your binary—avoid it on **macOS**.

**Nmap `ncat`** (often clearer flags than legacy `nc`):

```
python3 generate_cot_payload.py | ncat -u --send-only 239.1.2.1 6969
```

If packets never show up on the segment, compare against **`socat`** above, confirm the right interface, and capture with e.g.
`sudo tcpdump -n 'host 239.1.2.1 and udp port 6969'`.

On **multi-homed** machines you may need to bind the **outgoing interface** (OS-specific: choose the NIC that reaches the multicast segment, or set `IP_MULTICAST_IF` in your own sender code). If packets do not appear on the wire, verify routing and that you are not sending on the wrong interface.

**Listening with `nc`:** a plain `nc -u -l 6969` bind usually does **not** join a multicast group, so you may not see multicast traffic. Prefer **`socat`** with `ip-add-membership` (below), **Wireshark/tcpdump**, or a small socket program that calls `IP_ADD_MEMBERSHIP`.

### Join the group and listen (quick sanity check)

Example receiver that joins `239.1.2.1` on port `6969` and prints datagrams:

```
socat UDP4-RECVFROM:6969,reuseaddr,ip-add-
        membership=239.1.2.1:0.0.0.0,fork -
```

Replace `0.0.0.0` with the **IP address of the interface** that should receive the group (e.g. `192.168.1.50`) when more than one interface exists. Syntax can vary slightly by `socat` version and OS; use `socat -h` or the man page for `ip-add-membership` details.

### Operational notes

- **MTU**: keep datagrams small enough for the path (often **≤ ~1400 bytes** payload if you cross Wi-Fi or unknown tunnels).
- **Beyond one subnet**: multicast routing needs IGMP/PIM (or similar); senders often set **TTL** (`IP_MULTICAST_TTL`) if traffic must cross routers—lab defaults are usually TTL 1 (link-local only).
- **Same-host loopback**: some stacks do not loop multicast back to a socket on the same host; use two machines, a VM, or capture with `tcpdump` / Wireshark on the interface for local tests.
- **Firewall**: allow inbound UDP on the port you use (e.g. `6969`) for listeners.
- **Security**: multicast is visible to every host on the segment that joins the group; do not send sensitive data on untrusted networks without appropriate controls (VPN, encrypted gateways, etc.).

---

# Python vs C++ (behavioral differences)

| Topic | Python | C++ |
|---|---|---|
| `--payload-json` | Parsed with `json`; invalid JSON exits with error; output compact JSON in `<remarks>`. | Opaque string → `<remarks>` (no validation). |
| `--human-readable` | Not defined (use `--pretty`). | Accepted as alias for `--pretty`. |
| Dependencies | Stdlib only | Stdlib + compiler only |

---

# Protobuf CoT (separate wire format)

Many TAK deployments also use **Protocol Buffers** for on-the-wire messages (often as part of a larger **TAK message** envelope, not "the same XML bytes in protobuf"). This repository's sample tools emit **XML only**. If you add protobuf support elsewhere, plan

for:

### 1. Obtain schema (`.proto`) files

Typical sources:

- **TAK / ATAK open-source trees** — protobuf definitions are maintained alongside the Android / server code; start from TAK-Product-Center and ATAK-CIV and search for `proto` / `protobuf` / `TakMessage` in the repository.
- **Ecosystem libraries** — packages such as `takproto` on PyPI bundle or reference schema versions; **pin a version** that matches your target ATAK / server build.

Always align **schema version** with the **software version** you interoperate with; protobuf is unforgiving of field-number skew.

### 2. High-level development path (any language)

1. **Lock the contract** — identify the exact message type (e.g. outer envelope + inner CoT payload) your consumer expects (mesh vs stream vs server APIs may differ).
2. **Vendor the protos** — copy `.proto` files into your repo or fetch as a submodule; record **version / commit** in your README.
3. **Generate code** — run `protoc` (or `buf generate`) to produce language bindings (**C++**, **Java**, **Python**, Go, etc.).
4. **Map fields** — translate your logical event (uid, type, times, lat/lon, detail…) into the **generated types**. Do not assume "paste XML into a string field" will work unless your schema explicitly defines that.
5. **Golden tests** — capture hex dumps or known-good messages from a reference client and add round-trip / byte-for-byte tests where feasible.
6. **Operational debugging** — use `protoc --decode=FULL.Message.Name` with the same `.proto` set to inspect binary payloads; for XML, use `xmllint` or your pretty-printer.

### 3. If you extended *these* sample apps for protobuf

- **Python:** add a dependency on `protobuf` (and optionally `takproto`), generate stubs, add a CLI flag such as `--wire-format xml|protobuf`, and branch encoding after building the logical event.
- **C++:** add `libprotobuf`, run `protoc` in the `Makefile`, link the generated `.cc` files, same CLI branching as above.

That work is intentionally **out of scope** for the current stdlib-only / header-only samples, but the steps above are the usual industry approach.

---

# README as PDF

A PDF copy of this document can be regenerated from the same directory:

```
make readme-pdf
```

**Requirements:** Pandoc and **Google Chrome** (Chromium-based headless print). On macOS the default Chrome path is used. On Linux, point `make` at your browser, for example:

```
make readme-pdf CHROME=/usr/bin/google-chrome
# or
make readme-pdf CHROME=/usr/bin/chromium
```

The output is **`README.pdf`** next to `README.md`. A temporary `README.html` is removed after the build.

---

# Disclaimer

These generators are **educational / test utilities**. They are not a complete implementation of every CoT profile, security model, or TAK Server feature. Validate all type strings, `how` values, and detail extensions against your operational requirements and receiving system documentation.